

How to design a system that satisfies the required features/behaviors?

System architecture diagram with high-level components/modules

For each module (class) discuss its functionality and interface (methods) (eg UML diagram)

Additional design could include algorithm (pseudocode), database (ER diagram, tables, keys), communication over network (protocol)

Table of Contents

1. Introduction

1.1. Purpose

1.2. Scope

2. System Design

2.1. Vision Transformer Model

2.2. Data Processing Pipeline

2.3. Deployment Platform

3. Algorithm Design

3.1.

1. Introduction

1.1. Purpose

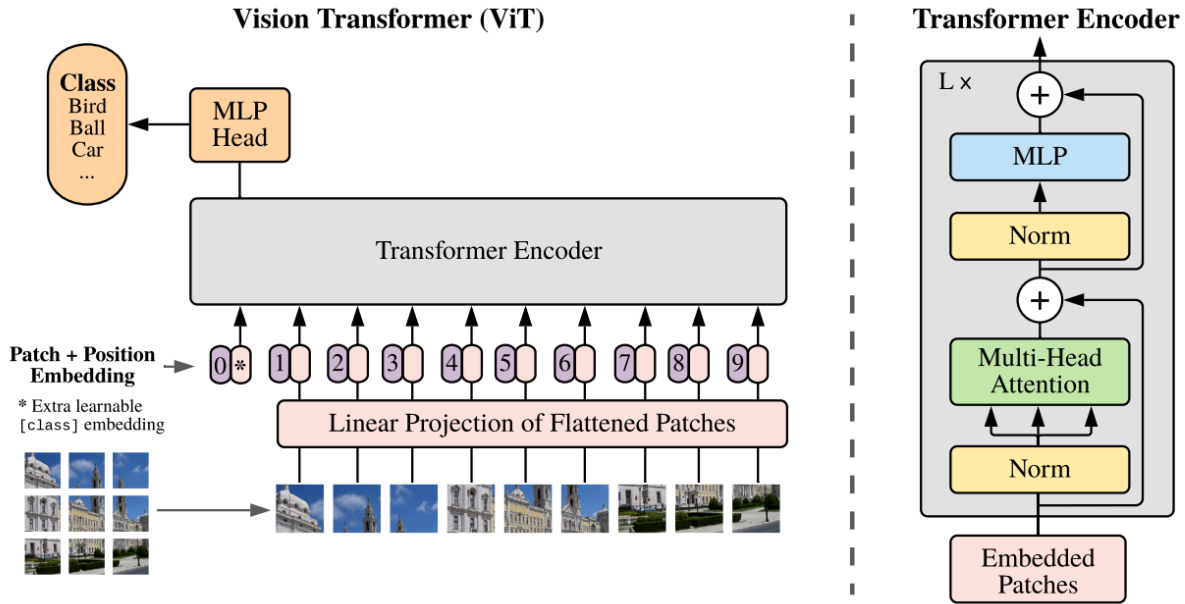
The purpose of this Design Document is to describe the architecture, components, and design decisions for the HALO system through system architecture diagrams and schematics.

1.2. Scope

This document covers the design and architecture of HALO, including its neural network model, data processing pipeline, and deployment platform.

2. System Design

2.1. Vision Transformer Model



* weights restricted to the set $\{0,1\}$

Image to Patches

- The $H \times W \times C$ image is split into non-overlapping $P \times P$ tiles. Each tile is flattened into a vector of length $P^2 C$. This converts the 2-D image into a sequence that the transformer can read. With patch size P , the sequence length is $N = \frac{HW}{P^2}$.
- 1-Bit: This can be implemented using stride- P convolution by storing weights in $\{0,1\}$ and bit-pack activations to cut memory and bandwidth.

Patch Embedding

- Each flattened patch in $\mathbb{R}^{P^2 C}$ is linearly projected to a D -dimensional token $x_i = W_{EPi}$, which produces an $N \times D$ matrix. This places all patches into a common feature space of width D for later comparisons.

- 1-Bit: Use a binary $W_E \in \{0, 1\}^{D \times P^2 C}$ and compute with bitwise AND + popcount, then multiply by a learned scale per output channel.

CLS Token

- A learned vector $x_{cls} \in \mathbb{R}^D$ is prepended, which makes the sequence length $N + 1$. This token aggregates global information that the classifier reads at the end.

Position Embeddings

- A position code p_i is added to every token in order to restore patch layout as Transformers do not know order by default.

Embedded Token Sequence

- The standardized sequence $X \in \mathbb{R}^{(N+1) \times D} : [x_{cls}, x_1, \dots, x_N] + pos$ is derived; this is the input to all encoder blocks.